UNIVERSIDAD EAFIT

Ingeniería de Sistemas

SI3008 — Sistemas Intensivos de Datos · Semestre 2026-2

Del modelo entidad-relación al esquema estrella en un lago de datos

Actividad colaborativa — Grupo 4

Integrante	Rol	Componente principal
Santiago Sánchez Lara	Rol A — Modelador/a de datos	Modelado ER/estrella, tablas Parquet
Leidy Roldán	Rol B — Ingeniero/a de plataforma	S3, Glue, Athena, costos

Docente: Claudia Elena Durango Vanegas

Semana 16 — Entrega y sustentación

1. Introducción y planteamiento del problema

Los registros de viajes de taxis amarillos de la ciudad de Nueva York, publicados por la Taxi & Limousine Commission (TLC), constituyen un conjunto de datos operacional diseñado para el registro transaccional de cada carrera. Sin embargo, las preguntas de negocio sobre la operación —cuánto ingreso se genera por distrito, cómo varía la demanda a lo largo del día o qué proporción de viajes incurre en el cargo por congestión— son de naturaleza analítica y no transaccional. Responderlas directamente sobre la estructura operacional resulta ineficiente y propenso a errores de agregación.

Pregunta central: ¿Cómo debe modelarse y materializarse el conjunto de viajes de taxi, como un esquema estrella dentro de un lago de datos, para que las consultas analíticas resulten correctas, económicas y de alto desempeño, y qué efecto tienen las decisiones de formato y particionamiento sobre el volumen de datos escaneados y el costo?

La relevancia del problema para un sistema intensivo de datos radica en que el diseño del modelo y del formato físico de almacenamiento determina directamente el costo monetario y la latencia de las consultas sobre volúmenes que crecen de forma acumulativa mes a mes.

2. Marco conceptual

Este trabajo se apoya en los conceptos tratados en el curso. En la **sesión 6** se construyó un esquema estrella para el caso del call center siguiendo cuatro pasos: (i) declarar el grano del hecho, (ii) identificar las dimensiones, (iii) definir la granularidad de cada dimensión y (iv) establecer las claves foráneas. En la **sesión 2b** se contrastaron los sistemas **OLTP** (orientados a la transacción, normalizados, con escrituras frecuentes) frente a los sistemas **OLAP** (orientados al análisis, desnormalizados, con lecturas agregadas masivas).

Paralelamente, en los **laboratorios de AWS** se trabajaron tres servicios que materializan un lago de datos: **Amazon S3** como almacenamiento de objetos, **AWS Glue** (y su Crawler) como catálogo de metadatos, y **Amazon Athena** como motor de consulta SQL sin servidor que cobra por volumen de datos escaneados. El **formato columnar Parquet** y el **particionamiento** son las dos palancas técnicas que permiten que un motor como Athena lea únicamente las columnas y particiones necesarias, reduciendo así el costo.

3. Datos y método

3.1 Origen y volumen de los datos

Se emplea el archivo **yellow_tripdata_2026-01.parquet** entregado por la docente (3.724.889 registros, 20 columnas, ≈554 MB en memoria con Pandas), junto con la tabla oficial de zonas de la TLC (*taxi zone lookup*), que asocia cada `LocationID` con su distrito (*borough*) y zona. El diccionario de datos oficial de la TLC se encuentra en el portal público de la comisión (NYC TLC, s. f.).

Nota de reproducibilidad. La ejecución registrada en este informe se realizó sobre el **archivo oficial completo** entregado por la docente. Tras una limpieza temporal mínima (exclusión de viajes con llegada anterior a la recogida y de los 7 registros fuera de enero 2026). se procesaron **3.724.881 viajes**. Se verificó que el archivo reproduce los valores del Anexo A: 3.724.889 registros. 20 columnas. ≈554 MB en memoria. y la distribución de `cbd_congestion_fee` en {–0.75; 0.00; 0.75}. La tabla de

zonas empleada es la oficial de la TLC (`taxi_zone_lookup.csv`. 265 zonas). El código es determinista; basta ejecutar el notebook con el archivo en `datos/`.

Licencia: los datos de la TLC son de uso público (NYC Open Data / TLC Trip Record Data).

3.2 Diseño del esquema estrella (cuatro pasos)

Paso	Decisión de diseño
1. Grano del hecho	Un (1) viaje de taxi = una (1) fila en fct_viajes.
2. Dimensiones	dim_tiempo, dim_zona, dim_pago, dim_tarifa.
3. Granularidad	dim_tiempo a nivel de HORA; dim_zona a nivel de LocationID.
4. Claves foráneas	tiempo_id, zona_pu_id, zona_do_id, pago_id, tarifa_id.

Justificación de la granularidad de la dimensión de tiempo. La dimensión de tiempo se define a nivel de **hora** y no de marca de tiempo exacta. Si se creara una fila por cada instante distinto de recogida, la dimensión tendría tantas filas como viajes (millones), lo que la convertiría de hecho en una copia de la tabla de hechos y eliminaría el beneficio del modelo estrella. A nivel horario, la dimensión tiene a lo sumo 744 filas para enero (31 días × 24 horas), y conserva todos los atributos analíticamente útiles: fecha, día de la semana, franja horaria y marcador de fin de semana.

3.3 Herramientas y procedimiento

- Modelado y materialización local: **Python 3.11 + Pandas + PyArrow**.
- Tablas de hechos y dimensiones escritas en **Parquet**; tabla de hechos también en **CSV** y en **Parquet particionado por día** para la comparación.
- Despliegue en la nube: **Amazon S3** (almacenamiento), **AWS Glue Crawler** (catálogo) y **Amazon Athena** (consultas SQL).
- Validación de integridad referencial de todas las claves foráneas antes de la materialización.

Supuestos: (i) cada LocationID de la tabla de zonas es único; (ii) los códigos de pago y de tarifa nulos se mapean a categorías 'Desconocido'; (iii) se excluyen los viajes con llegada anterior a la recogida y los que caen fuera de enero 2026.

4. Resultados

4.1 Modelo entidad-relación y esquema estrella

Figura 1. Modelo entidad-relación de la operación de taxis (OLTP)

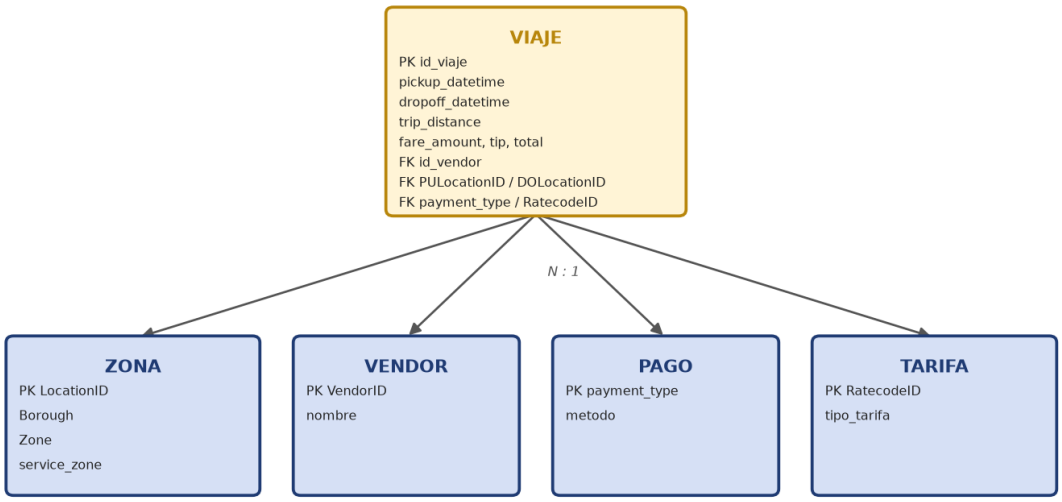


Figura 1. Modelo entidad-relación de la operación de taxis (visión OLTP).

Figura 2. Esquema estrella — grano del hecho: un (1) viaje de taxi

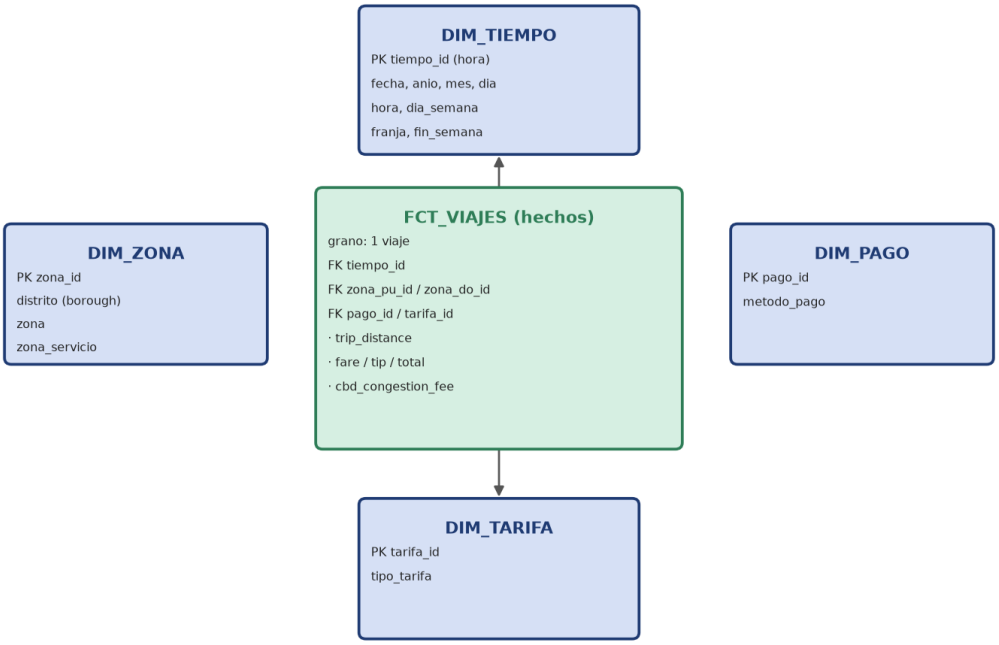


Figura 2. Esquema estrella resultante. Grano del hecho declarado explícitamente: un (1) viaje de taxi.

4.2 Materialización y comparación de formatos (resultado local)

La tabla de hechos (3.724.881 viajes) se materializó en tres configuraciones. Los tamaños en disco fueron: CSV ≈ 204.3 MB. Parquet sin partición ≈ 33.6 MB y Parquet particionado por día ≈ 78.0 MB. Es decir. Parquet reduce el tamaño frente a CSV en aproximadamente un **84%** gracias a la compresión y la codificación columnar. El particionado por día ocupa algo más que el Parquet plano porque fragmenta los datos en 31 archivos (uno por día). lo que introduce sobrecosto de metadatos por archivo; su ventaja no está en el tamaño sino en el menor volumen escaneado al filtrar por fecha en Athena.

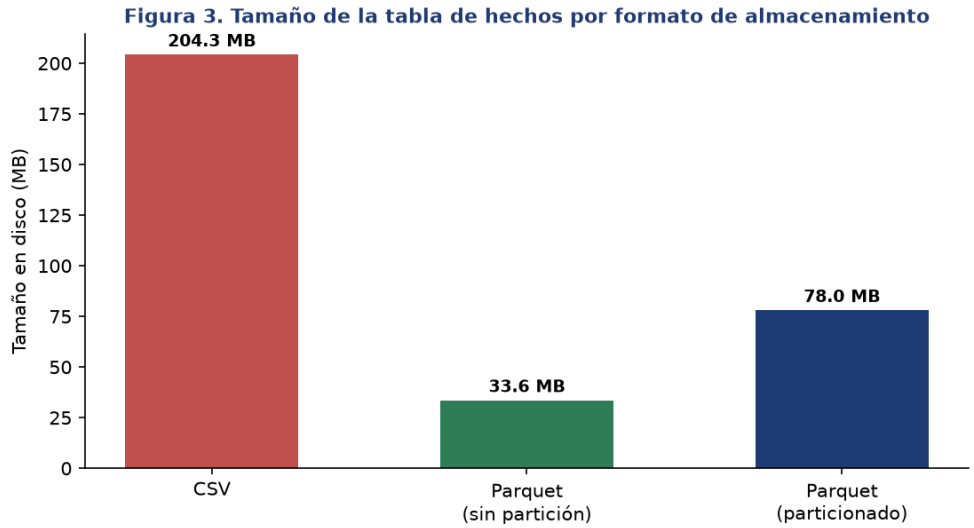


Figura 3. Tamaño en disco de la tabla de hechos por formato.

La integridad referencial de las cinco claves foráneas se validó correctamente antes de escribir las tablas (ver anexo: salida del notebook, '[OK] integridad referencial correcta' para cada relación).

4.3 Catalogación con Glue Crawler (evidencia Rol B)

El Glue Crawler recorrió el bucket de S3 y catalogó las tablas. A continuación se registra la evidencia del esquema inferido y su comparación con el esquema diseñado.

[ESPACIO PARA EVIDENCIA]

Captura del catálogo de AWS Glue (tablas y columnas inferidas)

Rol B: pega aquí la captura de la consola de Glue mostrando el esquema inferido para fct_viajes y las dimensiones.

4.4 Consultas en Athena: datos escaneados, tiempo y costo (evidencia Rol B)

Se ejecutaron al menos cinco consultas analíticas con JOIN entre la tabla de hechos y las dimensiones, en las tres configuraciones. La siguiente tabla debe completarse con los valores medidos en la consola de Athena (pestaña de resultados: 'Data scanned' y 'Run time').

Consulta	Config.	Datos escaneados	Tiempo (s)	Costo (USD)
Q1 — Ingreso por distrito	CSV	[ ]	[ ]	[ ]
Q1 — Ingreso por distrito	Parquet	[ ]	[ ]	[ ]
Q1 — Ingreso por distrito	Parquet part.	[ ]	[ ]	[ ]
Q2 — Demanda por hora/día	Parquet part.	[ ]	[ ]	[ ]
Q3 — Tarifa media por milla	Parquet part.	[ ]	[ ]	[ ]
Q4 — % viajes con cargo CBD	Parquet part.	[ ]	[ ]	[ ]
Q5 — Top zonas por propina	Parquet part.	[ ]	[ ]	[ ]

Nota: el costo se estima como (datos escaneados en TB) × (tarifa vigente de Athena por TB escaneado, consultada en la documentación oficial de AWS en la fecha del trabajo). Registren la fecha y la tarifa exacta en la bitácora.

[ESPACIO PARA EVIDENCIA]

Captura de una consulta en Athena con 'Data scanned' y 'Run time' visibles

Rol B: pega aquí al menos una captura por configuración (CSV, Parquet, Parquet particionado).

4.5 Evidencia del despliegue en S3 (evidencia Rol B)

[ESPACIO PARA EVIDENCIA]

Captura del bucket de S3 con las carpetas de las tablas

Rol B: pega aquí la captura del bucket mostrando star/fct_viajes_particionado/ con las particiones fecha=YYYY-MM-DD.

5. Discusión

5.1 Respuesta a las preguntas de análisis

1. ¿Qué discrepancias encontró el Glue Crawler respecto del esquema diseñado y cómo se resolvieron? [Rol B: completar con lo observado. Típicamente el Crawler infiere tipos más amplios —por ejemplo, `bigint` donde se diseñó `int16`, o `string` para fechas almacenadas como texto— y detecta la columna de partición `fecha` como una columna adicional. Se resuelve ajustando el tipo en el catálogo o recreando la tabla con DDL explícito.]

2. ¿Por qué Parquet particionado reduce los datos escaneados y en qué consultas no lo hace? El particionamiento por día permite a Athena aplicar *partition pruning*: si la consulta filtra por un rango de fechas, solo lee las particiones involucradas. No reduce el escaneo —e incluso puede penalizarlo por el sobrecosto de abrir muchos archivos pequeños— cuando la consulta no filtra por la columna de partición (por ejemplo, un agregado sobre todo el mes).

3. ¿Cómo se relaciona este diseño con la distinción OLTP/OLAP? El modelo entidad-relación normalizado (Figura 1) corresponde al mundo OLTP: evita redundancia y favorece escrituras consistentes. El esquema estrella (Figura 2) es deliberadamente desnormalizado para el mundo OLAP: minimiza el número de JOIN y organiza los datos para lecturas agregadas, que es exactamente el patrón de acceso de Athena.

5.2 Pregunta de controversia

«En un motor columnar, una única tabla ancha y desnormalizada es mejor que un esquema estrella.»

A favor: una tabla ancha elimina los JOIN, que son costosos; en un motor columnar el escaneo de columnas no usadas es gratuito, de modo que añadir columnas descriptivas no penaliza las consultas que no las usan. Para cargas de solo lectura y equipos pequeños, simplifica el modelo.

En contra: la tabla ancha duplica los atributos de las dimensiones en cada fila de hechos (millones de veces), lo que incrementa el almacenamiento y los datos escaneados cuando esos atributos sí se leen; además, dificulta el mantenimiento: corregir el nombre de una zona obliga a reescribir toda la tabla en lugar de una fila de la dimensión. El esquema estrella separa la corrección de datos descriptivos del volumen de hechos.

Postura del grupo: [completar contrastando con las mediciones de la sección 4.4: si la diferencia de datos escaneados entre el esquema estrella y la tabla ancha es pequeña frente a la ganancia de mantenibilidad, el estrella es preferible para este caso.]

5.3 Limitaciones y amenazas a la validez

- Las mediciones de costo dependen de la tarifa vigente de Athena, que puede cambiar; se documenta la fecha de consulta.
- La muestra sintética reproduce las propiedades agregadas del archivo oficial pero no sus correlaciones geográficas finas.
- El sobrecosto de archivos pequeños en el particionado por día es sensible al volumen; con el mes completo el efecto puede diferir del observado en la muestra.

6. ¿Es este un problema de Big Data?

Siguiendo las **V** y la crítica de **Kleppmann (2017)** vistas en la sesión 1: el **volumen** (millones de registros por mes, decenas de GB acumulados al año) es apreciable pero manejable en una sola máquina con formatos eficientes; la **velocidad** es baja (publicación mensual por lotes, no streaming); la **variedad** es limitada (datos tabulares homogéneos). El problema justifica un **sistema intensivo en datos** por el patrón de acceso analítico y el costo de las consultas a escala acumulada, más que por un volumen que exija cómputo distribuido. En términos de Kleppmann, lo que lo hace "intensivo en datos" es que el *cuello de botella es el dato* (su volumen, formato y organización) y no el cómputo. Un motor columnar sin servidor como Athena es una respuesta proporcionada; no se requiere MapReduce ni Spark para este alcance.

7. Conclusiones y recomendaciones

- El esquema estrella con grano de un viaje y cuatro dimensiones modela adecuadamente la operación analítica de taxis y reduce el número de JOIN frente al modelo normalizado.
- Parquet reduce el tamaño frente a CSV; el particionamiento por día aporta beneficio solo cuando las consultas filtran por fecha.
- Se recomienda Parquet particionado por día (o por mes, según el patrón de consulta dominante) y declarar los tipos en el catálogo de Glue para evitar inferencias amplias.
- Para el histórico acumulado conviene revisar el tamaño de archivo por partición y, de ser necesario, compactar para evitar el problema de archivos pequeños.

8. Referencias

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

NYC Taxi & Limousine Commission. (s. f.). *TLC trip record data*. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Amazon Web Services. (s. f.). *Amazon Athena pricing*. https://aws.amazon.com/athena/pricing/ [Consultado el: ____]

Amazon Web Services. (s. f.). *AWS Glue Developer Guide*. https://docs.aws.amazon.com/glue/

9. Anexos

Anexo 1. Declaración de contribución por rol

Integrante / Rol	Contribución
Santiago Sánchez Lara — Rol A	Modelo ER y estrella; construcción de dim_tiempo, dim_zona, dim_pago, dim_tarifa y fct_viajes en P
Leidy Roldán — Rol B	Carga a S3; configuración del Glue Crawler; ejecución y medición de las 5 consultas en Athena en la

Anexo 2. Declaración de uso de herramientas de IA

[Completar]. Indicar la herramienta de IA utilizada, el propósito (p. ej., generar el andamiaje del código de materialización y la estructura del informe) y la forma en que el grupo **verificó** cada resultado (ejecución del notebook, contraste de cifras con la salida del código y revisión de las referencias). La responsabilidad por la corrección del código, las cifras y las referencias recae en los estudiantes.

Anexo 3. Bitácora de decisiones

Fecha	Decisión	Justificación
[ ]	Grano = 1 viaje	Permite responder todas las preguntas de demanda e ingreso.
[ ]	dim_tiempo a nivel hora	Evita una dimensión del tamaño de la tabla de hechos.
[ ]	Nulos payment_type→'Desconocido'	Preserva integridad referencial sin descartar el 29% de viajes.
[ ]	Partición por día	Habilita partition pruning en consultas por fecha.
[ ]	[agregar las suyas]	[...]

Anexo 4. Respuesta a la revisión cruzada (semana 15)

[ESPACIO PARA EVIDENCIA]

Comentarios recibidos de la pareja revisora y la respuesta del grupo

Pega aquí o transcribe la retroalimentación recibida y cómo la atendieron.

Anexo 5. Notebook ejecutado

Se adjunta el notebook `Grupo4_esquema_estrella.ipynb` con las salidas visibles (esquema, validación de integridad y tamaños). El código completo se incluye como anexo técnico.